

03 — Python data types through a Case 1 profit equation

/ 03 — 结合 Case 1 利润方程理解 Python 数据类型

The question / 原始问题 (Question)

As a beginner, I want to understand Python's different kinds of variables, such as `float` and `tuple`, rather than just naming conventions. Can we cover the main data types, inspect their size where applicable, and connect them to a real equation and function from Case 1?

作为新手，我希望理解 Python 几乎全部常见的“变量种类”，例如 `float`、`tuple`，而不只是变量如何命名。能否系统介绍主要数据类型（data types），查看适用的大小信息（size），再结合 Case 1 的真实函数与方程逐行解析？

This chapter covers the main built-in value types, NumPy arrays, and a short tour of other objects found in research code. Python also permits user-defined and library-defined types, so there is no finite list of every possible type. Learn the numeric and container sections first; the binary and further-object sections are reference material.

本章覆盖主要内置值类型（built-in value types）、NumPy 数组（arrays），并简要介绍科研代码中的其他对象（objects）。Python 允许自定义类型和库类型，因此不存在“所有可能类型”的有限清单。建议先掌握数值和容器；二进制与扩展对象部分可作为速查。

Run the complete example with Python 3.9+ and NumPy: `python examples/03_python_data_types.py`. It also contains numbered Spyder cells. The small inputs are teaching data, not a reproduction of the paper's optimal decisions. The example was checked with Python 3.9.25 and NumPy 1.26.4.

完整示例需要 Python 3.9+ 和 NumPy。在仓库根目录运行 `python examples/03_python_data_types.py`，或在 Spyder 按编号执行代码单元（cells）。小规模输入是教学数据，并非论文最优决策的复现。示例已在 Python 3.9.25、NumPy 1.26.4 下检查。

1. Name, object, type, and value / 1. 变量名、对象、类型和值 (Name, object, type, value)

```
noise_std = 0.12
print(type(noise_std))    # <class 'float'>
print(noise_std)          # 0.12
```

`noise_std` is a name bound to an object; that object has type `float` and value `0.12`. `float` is itself the name of a built-in type, not a declaration attached to `noise_std`. Assigning `noise_std = "unknown"` later binds that name to a string. It does not convert the previous float object into a string.

`noise_std` 是绑定（binding）到对象的名字；对象的类型是 `float`，值是 `0.12`。`float` 本身是内置类型的名称，不是附着在变量名上的永久声明。之后写 `noise_std = "unknown"`，是把名字重新绑定到字符串（string），不是把原来的浮点对象变成字符串。

Use `type(value)` to inspect the exact type and `isinstance(value, float)` to ask whether an object belongs to a type or its subclasses. `isinstance(True, int)` is also true because `bool` subclasses `int`. Avoid using built-in names such as `list`, `str`, or `float` for your own variables: `list = [1, 2]` hides the constructor `list()` in that scope.

用 `type(value)` 查看确切类型；用 `isinstance(value, float)` 判断是否属于某类型或其子类（subclass）。由于 `bool` 是 `int` 的子类，`isinstance(True, int)` 也为真。不要将自己的变量命名为 `list`、`str` 或 `float`：例如 `list = [1, 2]` 会在该作用域（scope）遮蔽内置构造器（constructor）`list()`。

A type annotation such as `noise_std: float = 0.12` communicates an intended type. By itself, it neither converts the value nor enforces the type at runtime.

`noise_std: float = 0.12` 中的类型注解（type annotation）表达预期类型。单独使用注解不会自动转换数值，也不会在运行时强制检查类型。

2. Numbers, text, and missing values / 2. 数值、文本与空值（Numbers, text, missing values）

These objects do not allow their contents to be changed in place. Reassigning a name is still allowed.

以下对象不可原地修改（immutable）；但变量名仍可以重新赋值（reassignment）。

Type 类型 (type)	Create a value 创建示例 (creation)	Inspect or use it 查看或使用 (inspection / use)
int	<code>count = 3</code>	<code>count + 1</code> ; an integer count 整数 (integer) ; <code>count + 1</code>
float	<code>noise_std = 0.12</code>	<code>noise_std ** 2</code> ; a real-valued approximation 浮点数 (floating point) ; <code>noise_std ** 2</code>
complex	<code>z = 2 + 3j</code>	<code>z.real</code> , <code>z.imag</code> , <code>abs(z)</code> 复数; <code>z.real</code> 、 <code>z.imag</code> 、 <code>abs(z)</code>
bool	<code>converged = True</code>	<code>if converged:</code> ; a logical condition 布尔值; <code>if converged:</code>
str	<code>label = "Case 1"</code>	<code>label[0]</code> gives "C"; <code>len(label)</code> gives 6 字符串; <code>label[0]</code> 为 "C", 长度为 6
NoneType	<code>result = None</code>	<code>result is None</code> ; no result yet 空值; 用 <code>result is None</code> 判断

Python `float` normally uses binary double precision. Decimal fractions may not be exact: `0.1 + 0.2 == 0.3` is false. Use `math.isclose` or `np.isclose` with tolerances suited to the problem. `None` is not zero, an empty string, or NumPy's numerical nan; `type(None)` gives `NoneType`. Numerical nan is a floating-point value, and `np.isnan` checks for it.

Python `float` 通常使用二进制双精度（binary double precision）。十进制小数未必精确可表示，因此 `0.1 + 0.2 == 0.3` 为假。数值比较可使用 `math.isclose` 或 `np.isclose`，并根据问题选容差（tolerance）。`None` 不等于零、空字符串或数值缺失标记 nan；`type(None)` 得到 `NoneType`。数值 nan 是浮点值，可以用 `np.isnan` 检查。

English example:

```
float("0.12")      # 0.12: parse text
int(3.9)            # 3: truncate toward zero
str(3)              # "3": produce text
bool(0)             # False
bool("False")       # True: a nonempty string
```

中文示例:

```
float("0.12")      # 0.12
int(3.9)           # 3
str(3)             # "3"
bool(0)            # False
bool("False")      # True
```

These operations parse text, truncate toward zero (not rounding), produce text, test zero, and test a nonempty string. The string "False" is not automatically interpreted as the Boolean value False.

这些操作依次是解析文本、向零截断（不是四舍五入）、转换为文本、判断零的真假、判断非空字符串的真假。不要以为字符串 "False" 会自动解释为布尔假值。

len(3) and 0.12.shape are invalid. Scalar values are not one-element sequences. len("你好") is 2 Unicode code points; encoded byte length is a different quantity.

len(3) 和 0.12.shape 都不合法。标量 (scalar) 不是长度为 1 的序列。len("你好") 是 2 个 Unicode 码点 (code points) , 与编码后的字节长度 (byte length) 不同。

3. Sequences: list, tuple, and range / 3. 序列 (Sequences) : list、tuple、range

Type 类型 (type)	Construction and access 创建与访问 (creation / access)	Can its slots change? 能否改变元素位置上的内容?
list list 列表	q = [1.0, 2.0, 3.0]; q[0] q = [1.0, 2.0, 3.0]; q[0]	Yes: q[0] = 4.0, q.append(5.0) 可变: q[0] = 4.0、q.append(5.0)
tuple tuple 元组	bounds = (0.0, 6.0); bounds[1] bounds = (0.0, 6.0); bounds[1]	No reassignment of tuple slots 不可替换元组中的元素引用
range range 整数范围	indices = range(3); indices[0] indices = range(3); indices[0]	No; describes 0, 1, 2 without storing a list 不可变; 表示 0、1、2, 不创建完整列表

Indexing starts at zero. q[-1] selects the last item; q[0:2] selects the first two. len(q) counts top-level items. Lists and tuples can contain mixed types, for example ("seed", 5, True), though numerical calculations usually benefit from homogeneous arrays.

索引 (index) 从 0 开始。q[-1] 是最后一个元素; 切片 (slice) q[0:2] 取前两个。len(q) 只统计最外层元素数。列表和元组可以混合类型, 例如 ("seed", 5, True); 数值计算通常更适合元素类型统一的数组。

English example:

```
single = (3,)          # a one-item tuple
not_a_tuple = (3)      # an int
lower, upper = (0.0, 6.0) # unpack two values
values = list(range(3)) # [0, 1, 2]
```

中文示例：

```
single = (3,)           # tuple
not_a_tuple = (3)       # int
lower, upper = (0.0, 6.0) # unpacking
values = list(range(3))  # [0, 1, 2]
```

The comma makes a one-item tuple. In Q1, return x, y returns a tuple of two arrays, which the caller unpacks. An array's shape, such as (4, 3), is also a tuple.

单元素元组靠逗号区分，而非仅靠括号。lower, upper = ... 是解包 (unpacking)。01 中的 return x, y 返回包含两个数组的元组，再由调用方解包。数组的形状 (shape)，例如 (4, 3)，也是元组。

4. Mapping and sets / 4. 映射与集合 (Mapping and sets)

Type 类型 (type)	Construction 创建 (creation)	Access and purpose 访问及用途 (access / purpose)
dict dict 字典	cfg = {"seed": 5, "std": 0.12}	cfg["seed"]; named settings cfg["seed"]; 具名配置
set set 集合	ids = {1, 2, 2}	2 in ids; unique members 2 in ids; 唯一成员
frozenset frozenset 不可变集合	fixed = frozenset({1, 2})	2 in fixed; immutable set 2 in fixed; 固定成员集合

A dictionary maps unique, hashable keys to values; len(cfg) counts entries. cfg["seed"] = 6 updates a value. cfg.get("missing") returns None by default, while cfg["missing"] raises KeyError. Dictionaries preserve insertion order. Sets have no positional indexing or promised iteration order; ids[0] is invalid. Use ids.add(3) to modify a set.

字典将唯一且可哈希 (hashable) 的键 (key) 映射到值 (value)；len(cfg) 是条目数。cfg["seed"] = 6 更新数值。cfg.get("missing") 默认返回 None，而 cfg["missing"] 会抛出 KeyError。字典保留插入顺序 (insertion order)。集合不支持位置索引，也不保证迭代顺序；ids[0] 不合法，可用 ids.add(3) 添加成员。

{ } is an empty dictionary; set() is an empty set. Hashability means an object can safely serve as a dictionary key or set member under Python's hashing rules. Lists cannot; tuples can only when all their elements are hashable. Do not assume that every immutable container is hashable regardless of its contents.

{ } 是空字典，set() 才是空集合。可哈希表示对象符合 Python 的哈希规则，能够用作字典键或集合成员。列表不可以；元组只有在所有元素都可哈希时才可以。不能只看容器自身不可变，就认定它一定可哈希。

5. Binary values: useful when reading files or buffers / 5. 二进制类型 (Binary types)：文件和缓冲区

Type 类型 (type)	Example 示例 (example)	Access and mutability 访问与可变性 (access / mutability)
bytes	raw = b"ABC"	raw[0] == 65; immutable bytes raw[0] == 65; 不可变字节串

Type 类型 (type)	Example 示例 (example)	Access and mutability 访问与可变性 (access / mutability)
bytearray	buf = bytearray(b"ABC")	buf[0] = 90; mutable bytes buf[0] = 90; 可变字节数组
memoryview	view = memoryview(buf)	view[0] = 90 modifies the backing buffer view[0] = 90 会修改底层缓冲区

For these one-dimensional byte examples, len() is 3. raw.decode("utf-8") produces text; "ABC".encode("utf-8") produces bytes. A memory view exposes an existing buffer without copying its data. Writability depends on the backing object: a view over bytes is read-only. More general memory views can have their own shape, format, and nbytes.

在这些一维字节示例中，len() 均为 3。raw.decode("utf-8") 解码为文本；"ABC".encode("utf-8") 编码为字节。内存视图（memory view）直接访问已有缓冲区（buffer），无需复制数据。能否写入取决于底层对象：基于 bytes 的视图是只读的。更一般的内存视图还可以有 shape、format 和 nbytes。

6. Mutation, aliases, and copies / 6. 可变性、别名与复制 (Mutation, aliases, copies)

English example:

```
q = [1.0, 2.0]
alias = q
copied = q.copy()
alias[0] = 9.0
# q is now [9.0, 2.0]; copied is still [1.0, 2.0]
```

中文示例：

```
q = [1.0, 2.0]
alias = q
copied = q.copy()
alias[0] = 9.0
# q: [9.0, 2.0]; copied: [1.0, 2.0]
```

alias = q adds another name for the same list. It does not copy the list. A shallow copy creates a new outer container but shares references to nested objects. A tuple can hold a mutable object: t = ([1, 2],) permits t[0].append(3), even though replacing the tuple slot with t[0] = [] is forbidden.

alias = q 让两个名字指向同一个列表，不会复制列表。浅复制（shallow copy）只创建新的外层容器，内部嵌套对象仍可能共享。元组可以包含可变对象：t = ([1, 2],) 允许 t[0].append(3)，但不允许 t[0] = [] 替换元组中的元素引用。

For numeric NumPy arrays, basic slices normally share data with the original array, while a.copy() copies the data. Inspect np.shares_memory(a, b) when the relationship matters. An object-dtype array can still contain references to shared Python objects after a shallow array copy.

对于数值 NumPy 数组，基本切片（basic slicing）通常共享原始数据，而 a.copy() 复制数据。必要时用 np.shares_memory(a, b) 检查。对于对象类型数组（object-dtype array），复制数组后，元素仍可能引用相同的 Python 对象。

7. NumPy: type, dtype, shape, size, and len / 7. NumPy: type、dtype、shape、size、len

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
```

Inspection 检查 (inspection)	Result 结果 (result)	Meaning in this example 本例含义 (meaning)
type(a)	numpy.ndarray	Type of the whole array object 整个数组对象的类型
a.dtype	float64	Element storage type 元素存储类型 (element storage type)
a.shape	(2, 3)	Two rows, three columns 两行三列
a.ndim	2	Number of axes 轴数 (number of axes)
a.size	6	Total number of elements 总元素数
len(a)	2	Length of the first axis 第一条轴的长度
a.itemsize	8	Bytes per element 每个元素的字节数
a.nbytes	48	Element buffer bytes, excluding object overhead 元素缓冲区字节数, 不含对象管理开销

type(a[0, 0]) is np.float64; type(a[0, 0].item()) is Python float. A NumPy scalar and a zero-dimensional array differ: np.array(0.12) is an ndarray with shape == () and size == 1, but len() is invalid. float64, int64, and bool describe common element dtypes; they do not determine an array's shape.

type(a[0, 0]) 是 np.float64, 而 type(a[0, 0].item()) 是 Python float。NumPy 标量与零维数组 (zero-dimensional array) 也不同: np.array(0.12) 是 ndarray, shape == ()、size == 1, 但不能调用 len()。float64、int64、bool 是常见元素类型, 并不决定数组形状。

Lists do not have array shape or dtype. [1, 2] * 2 repeats a list; np.array([1, 2]) * 2 multiplies each element. Prefer explicit dtypes for reproducible storage choices; default integer widths can depend on the NumPy version and platform.

列表没有数组的 shape 或 dtype。[1, 2] * 2 重复列表, 而 np.array([1, 2]) * 2 是逐元素乘法 (elementwise multiplication)。需要固定存储类型时, 显式指定 dtype; 默认整数宽度可能随 NumPy 版本与平台变化。

8. The real Case 1 function / 8. Case 1 的真实函数 (Original function)

Source: Code_Python_rogp39/Case1-Production-Planning/ → comparative-study-production-planning_v2_revised_eng.ipynb, section **S6.1**, in the separate GaussianProcess-Integrated-Optimization repository. The following two functions retain the notebook's executable statements. Extra type demonstrations elsewhere in this chapter are teaching additions, not claims about types used by this function.

来源：另一个仓库 GaussianProcess-Integrated-Optimization 中的 Code_Python_rogp39/Case1-Production-Planning/ → comparative-study-production-planning_v2_revised_eng.ipynb, **S6.1**。下方两个函数保留原 Notebook 的可执行语句。其他类型示例是教学补充，不代表该函数实际使用了所有这些类型。

```
def true_price_fn(x):
    x = np.asarray(x, dtype=float)
    return np.exp(-x / 2.0)

def realized_profit_samples(x, noise_matrix, cost_vec):
    x = np.asarray(x, dtype=float).reshape(1, -1)
    cost_vec = np.asarray(cost_vec, dtype=float).reshape(1, -1)
    realized_price = true_price_fn(x) + noise_matrix
    return np.sum(x * (realized_price - cost_vec), axis=1)
```

The equation below uses plain text so it renders identically in Markdown and offline PDF:

方程采用纯文本表示，使 Markdown 与离线 PDF 的显示一致：

```
price[s, j] = exp(-x[j] / 2) + noise[s, j]
profit[s] = SUM over j of x[j] * (price[s, j] - cost[j])
```

s is the scenario index and j is the production component index. Each scenario profit sums production quantity times price minus unit cost across components.

s 是情景索引 (scenario index) , j 是生产分量索引 (production component index) 。每个情景的利润，就是各项“生产量 × (价格 - 单位成本)”之和。

Python name Python 名称 (name)	Role in the equation 方程中的角色 (role)	Shape after conversion 转换后的形状 (shape)
x	Production quantities x[j] 生产量 x[j]	(1, J)
cost_vec	Unit costs cost[j] 单位成本 cost[j]	(1, J)
noise_matrix	Price errors noise[s, j] 价格噪声 noise[s, j]	(S, J) expected 预期 (S, J)
realized_price	Scenario prices 各情景的实际价格	(S, J)
Returned array 返回数组 (returned array)	One profit per scenario 每个情景一个利润	(S,)

`x`, `noise_matrix`, and `cost_vec` are parameters; `realized_price` is a local name. On the first line, `np.asarray(..., dtype=float)` converts a list or tuple to a floating-point array, or may reuse compatible array storage. `reshape(1, -1)` creates one row and infers the column count; `-1` is not negative indexing here. Local reassignment of `x` does not rebind the caller's `x_input` name.

`x`、`noise_matrix`、`cost_vec` 是参数 (parameters)，`realized_price` 是局部变量名 (local name)。第一行 `np.asarray(..., dtype=float)` 把列表或元组转换为浮点数组，也可能复用已有兼容数组的存储。`reshape(1, -1)` 设为一行并自动推断列数；这里的 `-1` 不是负索引 (negative indexing)。函数内部重新绑定 `x` 不会重新绑定调用方的 `x_input`。

The helper computes the noiseless price element by element. Adding the (S, J) noise matrix broadcasts the $(1, J)$ prices across scenarios. Subtracting costs and multiplying quantities is elementwise. Finally, `axis=1` sums across columns, leaving one value for each row. Omitting `axis` would combine all scenarios into one scalar.

辅助函数逐元素计算无噪声价格。加上 (S, J) 噪声时，形状为 $(1, J)$ 的价格沿情景方向广播 (broadcasting)。减成本、乘生产量都是逐元素运算。最后 `axis=1` 沿列求和，每行留下一个利润。如果省略 `axis`，则会把所有情景一起求和为一个标量。

`float` is a type passed as an argument; `dtype` and `axis` are keyword parameter names; `reshape` is a method. These are different roles despite appearing on the same line.

这一行中，`float` 是作为参数传入的类型，`dtype` 与 `axis` 是关键字参数名 (keyword parameter names)，`reshape` 是方法 (method)。它们出现在相同语句里，但角色不同。

The original function assumes compatible numeric inputs; it does not validate dimensions or enforce economic constraints. In particular, a $(S, 1)$ noise array can broadcast the same shock across every component. Verify the intended (S, J) shape before calling it; successful broadcasting alone does not prove the model is correct.

原函数假定输入为兼容的数值数据，没有验证维度或经济约束。特别是 $(S, 1)$ 噪声也可能成功广播，却会把每个情景的同一个扰动用于所有生产分量。调用前应核对预期形状 (S, J) ；能够运行不等于模型含义正确。

9. A hand-checkable run and Spyder inspection / 9. 可手算的案例与 Spyder 检查 (Worked run and inspection)

The script uses three quantities $[1.0, 2.0, 3.0]$, costs $(0.1, 0.2, 0.3)$, and four explicit noise rows:

脚本使用生产量 $[1.0, 2.0, 3.0]$ 、成本 $(0.1, 0.2, 0.3)$ ，以及四行情景噪声：

```
[ 0.00,  0.00,  0.00]
[ 0.10,  0.00, -0.10]
[-0.10,  0.10,  0.00]
[ 0.02, -0.03,  0.04]
```

The zero-noise profit is approximately 0.61168002. Relative to it, the other scenarios change profit by -0.20 , $+0.10$, and $+0.08$: multiply each noise row by the three quantities and add. Expected output:

无噪声利润约为 0.61168002。其余情景的利润变化分别为 -0.20 、 $+0.10$ 、 $+0.08$ ：将每行噪声与三个生产量对应相乘再求和即可核对。预期输出 (expected output)：


```
profits = [0.61168002 0.41168002 0.71168002 0.69168002]
profit shape = (4,)
Verification passed.
```

1. Run cells 1–3 to inspect scalar and container names in Spyder's Variable Explorer. The `samples` dictionary collects all 15 core types in one place. Some object types may be filtered or lack a graphical editor; `type(value)` in the console remains useful.

执行代码单元 1–3，在变量浏览器（Variable Explorer）查看标量和容器。`samples` 字典集中保存了 15 种核心类型。有些对象可能被过滤或没有图形编辑器，此时仍可在控制台用 `type(value)` 检查。

2. Place a breakpoint on the return line marked P1, then debug the script. At that point, the preceding assignments have executed and the return has not.

在标记 P1 的返回语句上设置断点（breakpoint），调试脚本。暂停时前面的赋值已完成，返回语句尚未执行。

3. Inspect `type(x)`, `x.dtype`, `x.shape`, `noise_matrix.shape`, `realized_price.shape`, and `type(x.shape)`. Expect (1, 3), (4, 3), (4, 3), and a tuple for the shape object.

检查 `type(x)`、`x.dtype`、`x.shape`、`noise_matrix.shape`、`realized_price.shape`、`type(x.shape)`。对应形状应为 (1, 3)、(4, 3)、(4, 3)，而形状对象自身是元组。

4. After returning, inspect `profits` and compare with `reference`, calculated using nested loops rather than broadcasting. The script checks their agreement and the scenario profit changes.

返回后查看 `profits`，与使用嵌套循环（nested loops）计算的 `reference` 比较。脚本验证两种计算结果一致，并检查每个情景的利润变化。

10. Other objects you will meet / 10. 科研代码中的其他对象（Further objects）

This is a recognition guide, not a requirement to learn every advanced class now. The final script cell creates inspectable examples without writing files or raising exceptions.

本节用于识别，不要求立即掌握所有高级类型。脚本最后一个单元创建可检查对象，不写文件，也不主动抛异常。

Object family 对象类别 (family)	Example 示例 (example)	What to recognize 需要识别的特点
Exact decimal / rational 十进制 / 有理数	<code>Decimal("0.1")</code> , <code>Fraction(1, 3)</code> <code>Decimal("0.1")</code> 、 <code>Fraction(1, 3)</code>	Standard-library numeric classes; not built-in float 标准库数值类，不是内置 float
File-system path 文件路径 (path)	<code>Path("results")</code>	Library object representing a path 表示路径的库对象
Function / module / class 函数 / 模块 / 类	<code>true_price_fn</code> , <code>np</code> , <code>float</code> <code>true_price_fn</code> 、 <code>np</code> 、 <code>float</code>	Callable function, imported module, type object 函数对象、模块对象、类型对象
Iterator / generator 迭代器 / 生成器	<code>iter([1, 2])</code> , <code>(i*i for i in range(3))</code> <code>iter([1, 2])</code> 、 <code>(i*i for i in range(3))</code>	<code>next()</code> consumes values; usually no <code>len()</code> <code>next()</code> 消耗元素，通常没有 <code>len()</code>

Object family 对象类别 (family)	Example 示例 (example)	What to recognize 需要识别的特点
Slice 切片 (slice)	<code>slice(0, 2)</code>	Reusable indexing selection 可复用的索引选择
Exception instance 异常实例 (exception)	<code>ValueError("bad input")</code>	An object that can be raised with <code>raise</code> 可以用 <code>raise</code> 抛出的对象
Special singleton values 特殊单例值 (singletons)	<code>Ellipsis, NotImplemented</code> <code>Ellipsis, NotImplemented</code>	Extended slicing; unsupported operator protocol 扩展切片; 不支持某运算的协议返回值

`NotImplemented` is distinct from the exception `NotImplementedError`; it is not a generic missing-value marker. Files, custom class instances, pandas tables, and optimization model objects also have their own types. Inspect their type and documented interface instead of assuming they behave like an array.

`NotImplemented` 与异常 `NotImplementedError` 不同，也不是通用空值标记。文件对象、自定义类实例、pandas 表格和优化模型对象都有自己的类型。应检查其类型及文档接口，不要假设它们都像数组一样使用。

11. Practice and source notes / 11. 小练习与资料 (Practice and sources)

Predict before running: What is the type of `(3)` versus `(3,)`? Why does `len(a)` differ from `a.size`? Why does assigning `alias = q` affect the original list later? Why does a profit array have shape `(4,)` instead of `(4, 3)`? The answers are in sections 3, 7, 6, and 8 respectively.

先预测再运行：`(3)` 与 `(3,)` 各是什么类型？为什么 `len(a)` 与 `a.size` 不同？为什么 `alias = q` 后修改别名会影响原列表？为什么利润数组是 `(4,)` 而不是 `(4, 3)`？答案分别见第 3、7、6、8 节。

The [Python built-in types reference](#)^[1] is the lookup source for numeric, sequence, mapping, set, and binary operations. The [NumPy ndarray reference](#)^[2] documents array attributes and methods. The worked inputs, manual profit comparison, and teaching organization here are specific to this handbook.

[Python 内置类型参考 \(built-in types\)](#)^[1] 可用于查阅数值、序列、映射、集合和二进制操作。[NumPy 数组参考 \(ndarray\)](#)^[2] 说明数组属性和方法。本章小规模输入、利润手算核对及教学安排为手册专门编写。

参考资料 (References)

- 1. <https://docs.python.org/3/library/stdtypes.html>
- 2. <https://numpy.org/doc/stable/reference/arrays.ndarray.html>